Northeastern University
Electrical and Computer Engineering

EECE 2560: Fundamentals of Engineering Algorithms

Fall 2026

Mini Project: Airport Code Lookup Service with a Binary Search Tree

Learning Objectives:

- Build a binary search tree from a real data file by repeated insertion, and build a balanced tree from sorted data.
- Search, insert and delete in a BST, covering every insertion, search and deletion case.
- Use traversals (inorder, preorder, level order) and validate that a tree is a BST.
- Measure how the insertion order changes the height, and connect it to the O(h) running time of every operation.

The Real-World Problem

Every flight booking, boarding pass and baggage tag identifies an airport by its three-letter **IATA code** (BOS for Boston Logan, JFK for New York Kennedy, DAC for Dhaka). A travel company needs a fast **lookup service** that, given a code, returns the airport's name, city, country and location, and that stays correct while airports are **added** and **removed** from its database. Customer agents also need **alphabetical listings** and **range queries** such as "every code from BOS to BWI".

You will build this service on your own binary search tree, keyed by IATA code, and load it with real airport data.

The Data

Source: OpenFlights airport database, file `airports.dat` (openflights.org/data.php, GitHub repository jpatokal/openflights). Use the copy posted on the course page so that everyone works with the same version. The data is published under the Open Database License; credit "OpenFlights.org" in your README.

Format: one airport per line, comma-separated, text in double quotes, `\N` for a missing value. The fields are:

#	Field	Example (line for Boston)	Use in the project
1	Airport ID	3448	-
2	Name	General Edward Lawrence Logan International Airport	stored
3	City	Boston	stored
4	Country	United States	stored, used in Task 6
5	IATA code	BOS	**the BST key**
6	ICAO code	KBOS	stored
7	Latitude	42.36429977	stored
8	Longitude	-71.00520325	stored
9–14	Altitude, timezone, DST, tz name, type, source	…	ignore

> **Facts about the posted file (check your loader against them)**
> 7,698 lines in total. **6,072** airports have a 3-letter IATA code; the others have `\N` and must be **skipped**. All 6,072 codes are different. They cover 235 countries; 1,251 airports are in the United States.

What You Must Build

Write C++ with two classes: `Airport` (the record) and `AirportBST` (the tree, `root` private). Use loops of the form `for (int i = 0; i < n; i++)`. **Before every task, write the pseudocode for its main steps**, as in the lecture notes.

Task 1 Load the data and build the tree (insert)

(a) Read `airports.dat` line by line, split each line into fields (careful: names can contain commas inside quotes), skip lines whose IATA code is `\N`, and store the records in an array (or vector) of `Airport`.

(b) Build the BST by inserting the records **in file order**. Count how many keys were inserted and report any duplicate (rule I4).

(c) Report the **height** of the tree and the **average depth** of a node.

Task 2 Search

(a) `find CODE` prints the airport and the **search path** (every code compared) and the number of comparisons.

(b) Test every search case: the root, a leaf, a code deep in the tree, a code that does not exist (for example XYZ), a code smaller than every key and one larger than every key. Also print the minimum and maximum code.

Task 3 Delete

(a) `remove CODE` deletes an airport (for example when it closes) using the three deletion cases.

(b) Find, in **your** tree, one code for each case and test it: a leaf (D1), a node with only a left child, a node with only a right child (D2), a node with two children (D3) and the root. Print the level order of the top three levels before and after each delete.

(c) `add` a made-up airport (code QQQ, your own name and city) and show it can be found, then removed.

Task 4 Traversals and validation

(a) `list` prints every airport in alphabetical code order (inorder). Check that it is sorted.

(b) `save FILE` writes the codes in **preorder**; `load FILE` rebuilds the tree from that file. Show that the reloaded tree has the same shape (same height and level order).

(c) `validate` checks the BST property with the range (lo, hi) method after every insert and delete in your tests.

Task 5 Range queries

`range FROM TO` lists every airport with a code from FROM to TO, **without visiting subtrees that are outside the range**. Report how many nodes you visited. For example `range BOS BWI` must return 113 airports.

Task 6 The shape experiment

Build the tree in three ways from the same 6,072 codes and fill in the table: (1) file order, (2) sorted order, (3) balanced build from the sorted array (middle element as the root, a `rebalance` command). For each, also time 10,000 random searches.

Build order	Height	Average depth	Time for 10,000 searches	Your explanation
File order				
Sorted order				
Balanced (middle rule)				
Random shuffle (bonus)				

Expected heights for the posted file: file order **177**, sorted order **6,071** (a chain, n − 1), balanced **12** ($\lfloor \log_2 6072 \rfloor$). Explain why the file order is so much worse than a random shuffle (about 25 to 29). Hint: look at how the lines of the file are ordered.

Note: inserting 6,072 sorted keys recursively may overflow the call stack; write insert and search **iteratively** (with a while loop).

Milestones

Milestone	Goal	Checkpoint (done when…)
M1 — Data and build	pseudocode for all operations; loader; Airport and AirportBST classes; insert; height	6,072 airports loaded, 0 duplicates; inorder list is sorted
M2 — Search and delete	find with path, min/max, remove (all cases), add	every search case and every delete case passes; validate is true after each
M3 — Traversals and queries	list, save/load in preorder, range	reloaded tree has the same shape; range BOS BWI returns 113
M4 — Experiment and publish	shape experiment, README, demo	table of Task 6 filled in and explained; repository public

The instructor sets a due date for each milestone to fit the course calendar.

Deliverables for your GitHub profile

- A public repository named `airport-bst-cpp` with `src/`, `data/` (or a download script) and `docs/`.
- `docs/pseudocode.md`: the pseudocode for build, insert, search, delete, validate, range and the balanced build.
- A README with the problem, how to build and run (`g++ -std=c++17 -O2 src/*.cpp -o airports`), a sample session, the Task 6 table with your explanation, and the data credit to OpenFlights.org.
- A short screen recording or GIF of a session: find, range, remove of a two-child node, validate.

Sample session

```
> load data/airports.dat
Loaded 6072 airports (1626 lines skipped: no IATA code). Duplicates: 0
Height: 177   Average depth: 18.3
> find BOS
Path: GKA -> … -> BOS   (comparisons: 12)
BOS  General Edward Lawrence Logan International Airport, Boston, United States
> find XYZ
Path: GKA -> … -> (NULL)   not found
> range BOS BWI
113 airports, 127 nodes visited
> remove BOS
Deleted BOS (case D2: only a left child, BOR)   validate: OK
> rebalance
Height: 12   Average depth: 10.7
```

The numbers are the expected results for the posted file when the tree is built in file order (the dots stand for the full search paths). The count of nodes visited includes every node your range function looks at, so a slightly different pruning test can give a slightly different number.